



CS-317



Operating Systems

LECTURE 11



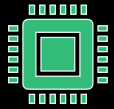
Concurrency

BOOK 1 CHAPTER 5

Central themes of OS design



Multiprogramming



Multiprocessing



Distributed
processing



Distributed processing example:

Clusters

- ▶ A cluster is a network of computers, all working on the same job.
- ▶ One computer in the cluster is called the master.
- ▶ All users submit jobs at the master node.
- ▶ The master node partitions the job, and sends the partitions to the slave nodes, asking them to work on their allocated partition.
- ▶ All slaves execute the same program, but the data is different.
- ▶ When the slaves have done their work, they send their partial results to the master.
- ▶ The master consolidates these results and send the final result to the user.

Concurrency



- ▶ Concurrency refers to the overlapping of life spans of a set of events, which may be similar or dissimilar.
- ▶ Concurrency can occur in many areas like:
 - ▶ communication among processes,
 - ▶ sharing of resources (such as memory, files, and I/O access),
 - ▶ synchronization of the activities of multiple processes, and
 - ▶ allocation of processor time to processes.

Contexts of concurrency

- ▶ **Multiple applications:** Multiprogramming allows processing time to be dynamically shared among several active applications.
- ▶ **Structured applications:** Some applications can be programmed as a set of concurrent processes.
- ▶ **Operating system structure:** The same structuring advantages apply to systems programs, and we have seen that operating systems are themselves often implemented as a set of processes or threads.



Principles of Concurrency

THE BASICS

Concurrent processing

- ▶ Concurrent processing of jobs happens when we employ:
 - ▶ Multiprogramming,
 - ▶ Time-sharing,
 - ▶ Multiprocessing,
 - ▶ Multithreading.
- ▶ The relative speed of execution of processes cannot be predicted.
- ▶ It depends on the activities of other processes, the way in which the OS handles interrupts, and the scheduling policies of the OS.



Difficulties for OS designer

- ▶ The sharing of global resources is fraught with peril.
- ▶ It is difficult for the OS to manage the allocation of resources optimally.
- ▶ It becomes very difficult to locate a programming error because results are typically not deterministic and reproducible.

since, multiple process sharing the resources, the bug made by which process is hard to find

A simple example

```
void echo( )  
{  
    chin = getchar( );  
    chout = chin;  
    putchar(chout);  
}
```

- ▶ This is a function called **echo**.
- ▶ Its body extends from one curly bracket to another.
- ▶ The keyword **void** shows it does not return a value.
- ▶ The **getchar** function captures a character from the keyboard and assigns it to variable **chin**.
- ▶ The second line of the function's body assigns the value of **chin** to a variable **chout**.
- ▶ The **putchar** function prints the value of **chout** onto the console output.

More about the function `echo`

echo process on multiprogramming

- ▶ We have a uniprocessor multiprogramming system supporting a single user.
- ▶ The user can jump from one application to another.
- ▶ Each application uses the same keyboard for input and the same screen for output.
- ▶ Each application needs to use the procedure `echo`.
- ▶ Only a single copy of the `echo` procedure is used, saving space.
- ▶ Such sharing of memory can lead to problems.

A problematic scenario

```
void echo( )  
{  
    chin = getchar( );  
    chout = chin;  
    putchar(chout);  
}
```

1. Process **P1** invokes **echo** and is interrupted immediately after **getchar** returns its value and stores it in **chin**. At this point, **chin** contains **x**.
2. Process **P2** is activated and invokes **echo**, which runs to conclusion, inputting and then displaying a single character, **y**, on the screen.
3. Process **P1** is resumed. By this time, the value **x** has been overwritten in **chin** and therefore lost. Instead, **chin** contains **y**, which is transferred to **chout** and displayed.



Resolution of the problem

```
void echo( )  
{  
    chin = getchar( );  
    chout = chin;  
    putchar(chout);  
}
```

1. Process **P1** invokes **echo** and is interrupted immediately after **getchar** returns its value and stores it in **chin**. At this point, **chin** contains **x**.
2. Process **P2** is activated and invokes **echo**. However, because **P1** is still inside the **echo** procedure, although currently not running, **P2** is blocked from entering the procedure.
3. Later, **P1** is resumed and completes execution of **echo**. The character **x** is displayed.
4. When **P1** exits **echo**, this unblocks **P2**.
5. When **P2** is later resumed, the **echo** procedure is successfully invoked.

On a multiprocessor...

	Process P1		Process P2
	•		•
x	chin = getchar();		•
	•	y	chin = getchar();
y	chout = chin;	y	chout = chin;
	putchar(chout);		•
	•		putchar(chout);
	•		•

y

y

- ▶ In a multiprocessor system, there is no mechanism for controlling access to the shared global variable.
- ▶ Processes P1 and P2 are both executing, each on a separate processor. Both invoke echo.
- ▶ Events on the same line take place in parallel.
- ▶ The character input to P1 is lost before being displayed, and the character input to P2 is displayed by both P1 and P2.

Resolution of the problem

```
void echo( )  
{  
    chin = getchar( );  
    chout = chin;  
    putchar(chout);  
}
```

- ▶ Processes **P1** and **P2** are both executing, each on a separate processor.
- ▶ **P1** invokes **echo**.
- ▶ While **P1** is inside **echo**, **P2** invokes **echo**. Because **P1** is still inside **echo**, **P2** is blocked from entering it.
- ▶ At a later time, process **P1** completes execution of **echo**, exits that procedure, and continues executing.
- ▶ Immediately upon the exit of **P1** from **echo**, **P2** is unblocked and when resumed, it begins executing **echo**.

Why does the problem occur?

multiprog/uni [r]cessor : due to interrupt
multiprocessor: due to sharing of resources

- ▶ In the case of a **uniprocessor system**, the reason we have a problem is that an **interrupt** can stop instruction execution anywhere in a process.
- ▶ In the case of a multiprocessor system, we have that same condition, and, in addition, a problem can be caused because **two processes may be executing simultaneously** and **both trying to access the same global variable**.
- ▶ The solution to both types of problem is the same: **control access to the shared resource**.



“If a resource is not shared between two or more processes, it cannot create a concurrency-related problem.”

So what we need to watch out for are shared resources!



“If a resource is not modifiable, it cannot create a concurrency-related problem.”



So what we need to watch out for are resources whose values can be written to or changed!

As a system designer, we
should be cautious about



Shared modifiable data



Race Condition



A race condition occurs when multiple processes or threads read and write data items so that the final result depends on the order of execution of instructions in the multiple processes.

Race Condition:

Example 1



- ▶ P1 and P2 share the global variable a .
- ▶ At some point in its execution, P1 updates a to the value 1. 
- ▶ At some point in its execution, P2 updates a to the value 2.
- ▶ The "loser" of the race determines the final value of a .

Race Condition:

Example 2

- ▶ P3 and P4 share global variables b and c , with initial values $b = 1$ and $c = 2$.
- ▶ At some point in its execution, P3 executes the assignment $b = b + c$.
- ▶ At some point in its execution, P4 executes the assignment $c = b + c$.
- ▶ The two processes update different variables.
- ▶ The final values of the two variables depend on the order in which the two processes execute these two assignments.
- ▶ If P3 executes its assignment statement first, then the final values are $b = 3$ and $c = 5$.
- ▶ If P4 executes its assignment statement first, then the final values are $b = 4$ and $c = 3$.



Homework:

Take some time to figure out the answer to this question.

P1 and P2 share global variables b and c , with initial values $b = 5$ and $c = 1$. At some point in its execution, P1 executes the assignment $b = b - c$. At some point in its execution, P2 executes the assignment $c = b - c$. What final values are possible for the two variables?

Operating System Concerns

- ▶ The OS must be able to keep track of the various processes.
- ▶ For each active process, the OS must allocate and deallocate:
 - ▶ Processor time,
 - ▶ Memory,
 - ▶ Files,
 - ▶ I/O devices.
- ▶ The OS must protect the data and physical resources of each process against unintended interference by other processes.
- ▶ The output of a process must be independent of the speed at which its execution is carried out relative to the speed of other concurrent processes.





Process Interaction

PROBLEMS ARISING OUT OF CONCURRENCY

How to classify process interaction?

- ▶ We can classify the ways in which processes interact based on the degree to which they are aware of each other's existence.
- ▶ Processes unaware of each other,
- ▶ Processes indirectly aware of each other,
- ▶ Processes directly aware of each other.

Processes unaware of each other

- ▶ These are independent processes that are not intended to work together.
- ▶ There is no exchange of information between these processes.
- ▶ Example: A batch of jobs in a multiprogramming uniprocessor system, each submitted by a different user.
- ▶ Although the processes are not working together, the OS needs to be concerned about competition for resources.

Competition

- ▶ The state of the resource for which the processes are competing remains unaffected after it has been acquired and used by a process.
- ▶ Examples of such resources include I/O devices, memory, processor time, and the clock. They are called reusable resources.
- ▶ If two processes both wish access to a single resource, then one process will be allocated that resource by the OS, and the other will have to wait. Therefore, the process that is denied access will have a larger turnaround time.

Problems of competing processes

- ▶ In the case of competing processes, we face three control problems:

- ▶ Mutual exclusion,



- ▶ Deadlock, and

- ▶ Starvation.

- ▶ The last two problems are a consequence of the first.

Lessons learned



- ▶ We were introduced to the concept of concurrency.
- ▶ We realized that sometimes concurrency is desirable and sometimes it is problematic.
- ▶ We realized that concurrency related problems always exist for shared modifiable data.
- ▶ We realized what race condition is.
- ▶ We were introduced to the types of process interaction.

Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.